

A surprise-examination (sorites) proof of Gödel’s second incompleteness theorem in Basic Recursive Arithmetic

Thierry Coquand

June 4, 2026

Abstract

We describe a second, independent machine-checked proof of Gödel’s second incompleteness theorem for Church’s Basic Recursive Arithmetic (BRA), following Chaitin’s information-theoretic route and the surprise-examination argument of Kritchman and Raz. The formal system, its numerals, the encoding $code(\cdot)$, the verifier **thmT** and the numeral functor **num** are exactly those of the companion paper [1], which gives a first proof of the same theorem along Guard’s lines; we do not repeat their construction here. The two ingredients specific to this proof are (i) an *internal* form of Chaitin’s first incompleteness theorem — a single BRA implication built around an object-level interpreter — and (ii) a *sorites*, or heap-of-sand, descent that turns Chaitin’s theorem into the second incompleteness theorem. The whole development is checked in Agda under `--safe --without-K --exact-split` with no postulates and no holes, from the single hypothesis that T does not prove $0 = 1$.

1 The statement

We work entirely inside the system T of Basic Recursive Arithmetic in the formulation of Guard, with Shoenfield’s notation; the axioms, the numerals $\mathbf{k}_n = \mathbf{s}^n(\mathbf{O})$, the encoding $code(\cdot)$, the verifier $\mathbf{thmT} \in Fun_1$ (with $\vdash \mathbf{thmT}(code(d)) = code(A)$ whenever d codes a derivation of A), the numeral functor $\mathbf{num} \in Fun_1$ (with $\vdash \mathbf{num}(\mathbf{k}_n) = code(\mathbf{k}_n)$) and the derivability-internalising Theorems 12 and 13 are all as set up in the companion paper [1], to which we refer for every detail of the system. We write $\vdash A$ (or $Deriv(A)$) for “ A is a theorem of T ”.

As in [1], the consistency of T is the *open* formula

$$Con_T \equiv \neg(\mathbf{thmT}(\mathbf{x}_0) = code(\mathbf{O} = \mathbf{s}\mathbf{O})),$$

with the single free variable $\mathbf{x}_0$. Since BRA has no object quantifiers, a theorem is an open formula read as implicitly universally quantified over its free variables, so Con_T says: *for every term $\mathbf{x}_0$, $\mathbf{x}_0$ does not code a derivation of $\mathbf{O} = \mathbf{s}\mathbf{O}$* . The theorem we prove is the same meta-implication that the Guard-route proof establishes,

$$\boxed{\vdash Con_T \implies \vdash (\mathbf{O} = \mathbf{s}\mathbf{O})},$$

i.e. if T proves its own consistency then T is inconsistent. The arrow $\implies$ is a meta-level implication between derivability judgements, realised in Agda as a total function $Deriv(Con_T) \rightarrow Deriv(\mathbf{O} = \mathbf{s}\mathbf{O})$. The consistency hypothesis is consumed exactly once per step of the descent of §3, and is the *only* assumption: every other ingredient is constructed.

2 Chaitin’s first incompleteness theorem, internalised

Chaitin’s proof of the first incompleteness theorem [2, 3] replaces the liar sentence by Berry’s paradox and Kolmogorov complexity. We need this argument not as a metatheorem but as a

single BRA implication, with the underlying universal machine realised as an honest object-level program. This section describes that realisation in ordinary computational terms; the detailed Agda construction is part of the development cited in §4.

2.1 Programs as ternary numbers

A *program* is simply a natural number. Read a number p in base 3; its digits, drawn from the three-symbol alphabet $\{1, 2, 3\}$, form a string, and that string is fed to a fixed parser **parse** that turns it into a syntax tree — an expression of the object language. The *length* $|p|$ of the program is the number of base-3 digits, so

$$|p| \leq n \iff p < 3^{n+1}.$$

Thus the enumeration of programs is the *identity*: the k -th program is the number k , and the finite set of programs of length at most L^* is exactly the initial segment

$$\{p : p < N\}, \quad N := 3^{L^*+1}.$$

This is the single simplification that makes the whole argument feasible: there is no separate enumeration to verify — “the set of short programs” *is* an interval of numbers, and quantification over it is bounded quantification over $\{p < N\}$.

2.2 The universal interpreter as a CK machine

The interpreter **U** that runs a parsed program is realised as a small abstract machine of the Landin CK kind. A *configuration* is a pair

$$\langle c, \kappa \rangle,$$

where c is the expression currently under evaluation (the *control*) and κ is a *continuation stack* recording the contexts still to be returned to. A one-step transition function **step** fires a single reduction rule: it either decomposes c , pushing the deferred work onto κ , or, when c has reached a value, pops the top frame of κ and plugs the value into it. Running the program for l steps is the l -fold iteration **step** ^{l} started from the initial configuration; the machine halts when the control is a value and the stack is empty, and the output is read off that final configuration. Around this evaluator we wrap one more layer, a bounded μ -search loop, so that the outer behaviour of **U** matches the informal recipe “enumerate **thmT**(0), **thmT**(1), ... until a fitting proof is found” used by the diagonal program below.

For a program p we write

$$\mathbf{def}_p(l, u) \equiv \text{“running } p \text{ for } l \text{ steps halts with output } u + 1\text{”}$$

(the $+1$ separates “halted with value u ” from “not yet halted”, coded by 0); $\mathbf{def}_p \in Fun_2$ for each p . Folding the bounded disjunction of the $\mathbf{def}_p$ over the finite set $\{p < N\}$ gives a single object function **CK** $\in Fun_2$ with

$$\mathbf{CK}(u, x) = \mathbf{O} \iff \text{some program } p < N, \text{ run for } x \text{ steps, outputs } u + 1,$$

i.e. $\mathbf{CK}(u, x) = \mathbf{O}$ expresses $K(u) \leq L^*$ at run-length x , and the Kolmogorov bound

$$K(u) > L^* \text{ is expressed by } \mathbf{CK}(u, x) \neq \mathbf{O}$$

with x a free run-length variable.

2.3 Term-valued fuel: a Gandy–Howard majorant

The delicate point is not the diagonal argument — which is a faithful rendering of Chaitin’s — but the fact that the interpreter must be run *inside* a **thmT**-fact. Since **thmT**, **U** and **step** all take object terms, the number of steps appearing in such a fact must itself be an object *term*, not a meta-level natural; and Theorem 13 of [1] internalises only statements about BRA terms. A completeness statement of the naive form “for every numeral $\bar{n}$ large enough, **step** ^{$\bar{n}$} on the program halts” is therefore useless: it is quantified over meta-naturals and cannot be fed into an internal derivation. What is needed instead is a completeness statement with a closed *combinator* in the fuel slot,

$$(x : \text{Term}) \rightarrow \vdash \left(\mathbf{step}^{\mathbf{fuel}(f)(x)}(\text{init}(f, x)) = \text{halt}(f \cdot x) \right),$$

where $\mathbf{fuel}(f) \in \text{Fun}_1$ is built once and for all from f .

Proving such a statement by induction on the combinator tree of f stalls at the primitive-recursion node, where one would need numerical control over the running time of the recursive calls — information not available at the derivation level. The resolution adapts the classical *majorisation* method of Howard and Gandy [6, 4]: one defines, by primitive recursion mirroring the structure of f , a closed combinator $\mathbf{fuel}(f)$ that provably *majorises* the actual number of steps any run of f consumes at every term input. The induction then carries the stable shape “the run completes at *any* $\text{fuel} \geq \mathbf{fuel}(f)(x)$ ”, which descends through the recursion node because $\mathbf{fuel}$ is itself recursive in step with f . The μ -search loop needs the same idea one level higher, with a majorant $\mathbf{fuel}_\mu$ stacked on top of $\mathbf{fuel}$ by a fuel-composition lemma. The closed term that finally occupies the fuel slot is $\mathbf{fuel}_\mu(\cdots) + \mathbf{fuel}(\cdots)$. Without this majorisation there would simply be no term to put there.

2.4 The diagonal program and the internal implication

Chaitin’s short program g_{L^*} enumerates $\mathbf{thmT}(0), \mathbf{thmT}(1), \dots$ until it meets the code of a statement of the form $K(?) > L^*$, and then outputs the witnessed number $?$. Its size is a constant plus the number of digits needed to write L^* , that is $c + \log L^*$, which is below L^* once L^* is large. The program g_{L^*} is obtained as a genuine one-variable object combinator $G \in \text{Fun}_1$ by bracket abstraction — an honest BRA program, not a meta-level term transformer — so that $G \cdot w$ is literally the constructed code, as a *proved* T -equation. Let $\mathbf{out}(w)$ be the object function that reads the witness off w . The internalised Chaitin theorem is the single BRA implication

$$\vdash \left(\mathbf{thmT}(w) = \text{code}(K(\mathbf{out}(w)) > L^*) \right) \supset \left(\mathbf{thmT}(G \cdot w) = \text{code}(\mathbf{O} = \mathbf{s} \mathbf{O}) \right), \quad (\text{C})$$

valid for *every* term w , with no consistency premise. In words: T proves that *if* $\mathbf{thmT}(w)$ is the incompressibility statement about w ’s own read-back, *then* the code $G \cdot w$ is a T -proof of $0 = 1$. The deduction lives entirely inside the object logic as an implication; the run of the interpreter on g_{L^*} , with the Term-fuel of §2.3, is what closes the positive leg of (C). This is the only place where the evaluator machinery is needed; the descent of the next section uses (C) purely as a black box.

3 The sorites descent

Chaitin’s theorem says that, for a large enough L^* , no statement $K(x) > L^*$ is provable in a consistent T , even though for each x exactly one of $K(x) \leq L^*$, $K(x) > L^*$ holds and the former, when true, is provable (by exhibiting and running the short program). Kritchman and Raz [8] observed that this tension can be wound into the second incompleteness theorem by an argument modelled on the surprise-examination paradox.

3.1 The counting argument

Fix L^* as in §2 and put $N = 3^{L^*+1}$. Let

$$m = \#\{u \in [0, N] : K(u) > L^*\}.$$

Because there are only N programs of length $\leq L^*$, at most N of the $N+1$ values in $[0, N]$ can be compressible, so at least one is incompressible: $m \geq 1$, and this is provable in T . Kritchman and Raz then prove, *by induction on i and inside T* , that $\vdash (m \geq i)$ for $i = 1, 2, \dots$. The induction step $\vdash (m \geq i) \Rightarrow \vdash (m \geq i+1)$ is exactly where Chaitin's theorem and one use of consistency enter (it is their displayed equations (1) for Chaitin and (2) for the Σ_1 -completeness of $K(\cdot) \leq L^*$). But $m \leq N+1$ outright, so the induction cannot run forever: once i passes $N+1$, the provable statement $m \geq i$ contradicts $m \leq N+1$, and T proves $0 = 1$.

3.2 Why this is a sorites, not a vicious circle

The original surprise-examination paradox (“there will be an exam next week but you will not know the day until it is held”) is usually analysed as a self-referential knowledge puzzle. The mathematical core that survives formalisation, however, has the shape of the *sorites*, or heap-of-sand, paradox. Think of the interval $[0, N]$ — equivalently the population of incompressible numbers counted by m — as a heap of sand. Each step of the induction removes one grain: it is, taken on its own, an entirely sound BRA derivation, consisting of one internal application of the Chaitin implication (C) reflected through a single use of consistency. No individual grain-removal is the culprit. Yet after $N+1$ removals the heap is gone and T has proved $\mathbf{O} = \mathbf{sO}$. The contradiction lives only in the iteration, exactly as in the heap paradox, and it is the second incompleteness theorem — the impossibility of T proving its own consistency — that explains why the seemingly innocuous step cannot in fact be taken all the way (cf. the resolution discussed in [8]: the missing premise is the consistency of T itself).

3.3 The formal realisation: six steps

In the formalisation the count m is carried by an open formula $S(r)$, indexed by a stage $r \in [0, N]$ and built as a negated big conjunction over “days”. For programs $p_r, \dots, p_N$ (each of length $\leq L^*$, ranging over $\{p < N\}$) write

$$K(\mathbf{x}_0; p_r, \dots, p_N) \equiv \bigwedge_{d=r}^N \mathbf{def}_{p_d}(\mathbf{x}_0, d),$$

“each program p_d describes the day d , at the common run-length $\mathbf{x}_0$ ”. The stage formula is

$$S(r) \equiv \vdash \neg K(\mathbf{x}_0; p_r, \dots, p_N) \quad (\text{for every choice of } p_r).$$

The descent climbs $S(0) \rightarrow S(1) \rightarrow \dots \rightarrow S(N)$, one grain at a time, and the inductive step $S(r) \rightarrow S(r+1)$ is the six-step block that realises Kritchman–Raz’s induction step; it follows the high-level note `clos` of the development and reuses the implication (C) as a black box.

Step 1 (run-monotonicity and the finite set). From $S(r)$ one derives the implication “ $K(\mathbf{x}_0; p_{r+1}, \dots, p_N) \neg \mathbf{def}_{p_r}(\mathbf{x}_1, r)$ ” at an *independent* run-length $\mathbf{x}_1$, the two fuels $\mathbf{x}_0, \mathbf{x}_1$ being aligned to a common bound by monotonicity of the interpreter in its step count (running longer never unsays a halt). Folding the bounded disjunction over the finite set $\{p < N\}$ rewrites the consequent as $K(r) > L^*$, giving

$$\vdash K(\mathbf{x}_0; p_{r+1}, \dots, p_N) \supset (K(r) > L^*).$$

Step 2 (encode and substitute the numeral). By Theorem 13 of [1] the Step-1 implication is internalised: one obtains a term f with $\vdash \mathbf{thmT}(f \cdot \mathbf{x}_0) = \mathit{code}(K(\mathbf{num} \mathbf{x}_0; p_{r+1}, \dots, p_N) \supset K(r) > L^*)$, the subject slot $\mathbf{x}_0$ being replaced inside the code by its numeral $\mathbf{num} \mathbf{x}_0$ (the consequent has no free $\mathbf{x}_0$ and is left inert).

Step 3 (internal modus ponens). An encoded modus ponens (the Carneiro-lifted `imp_encoded_mp` of [1]) chains Step 2 with a $\mathbf{thmT}$ -fact for the antecedent.

Step 4 (Theorem 13 for the conjunction). Writing the big conjunction as $K_r(\mathbf{x}_0) = \mathbf{O}$, the Σ_1 -completeness direction $(K_r(\mathbf{x}_0) = \mathbf{O}) \supset \mathbf{thmT}(D \mathbf{x}_0) = \mathit{code}(K(\mathbf{num} \mathbf{x}_0; p_{r+1}, \dots, p_N))$ supplies that antecedent fact (Kritchman–Raz’s equation (2)).

Step 5 (Chaitin). The internal Chaitin implication (C) is applied: $\vdash (\mathbf{thmT}(w) = \mathit{code}(K(r) > L^*)) \supset (\mathbf{thmT}(G \cdot w) = \mathit{code}(\mathbf{O} = \mathbf{s} \mathbf{O}))$. Composing Steps 3–5 yields $\vdash K_r(\mathbf{x}_0) = \mathbf{O} \supset (\mathbf{thmT}(h \cdot \mathbf{x}_0) = \mathit{code}(\mathbf{O} = \mathbf{s} \mathbf{O}))$.

Step 6 (consistency). Finally Con_T is instantiated at $h \cdot \mathbf{x}_0$ and, by contraposition, refutes the consequent — so the antecedent fails: $\vdash \neg K(\mathbf{x}_0; p_{r+1}, \dots, p_N)$, which is $S(r+1)$. This is the single use of consistency per grain (Kritchman–Raz’s equation (1)).

Base case. $S(0)$ is hypothesis-free and combinatorial. There are $N + 1$ days but only N program slots, so any assignment of a program to each day forces a collision $p_i = p_j$ with $i \neq j$; the two day-conjuncts then assert that one program, at the same run-length, outputs both $\mathbf{s}^i$ and $\mathbf{s}^j$, which is impossible. Hence $\neg K(\mathbf{x}_0; p_0, \dots, p_N)$.

Final clash. At the top stage $r = N$ the remaining conjunction is empty, hence provably true, so the Step-1 implication fires *unconditionally*: T proves $K(N) > L^*$, the Chaitin diagonal makes its provability into a proof of $0 = 1$, and Con_T refutes it. The heap is empty and

$$\vdash (\mathbf{O} = \mathbf{s} \mathbf{O}).$$

By external induction the chain $S(0) \rightarrow \dots \rightarrow S(N)$ closes the proof. This induction is not informal: it is carried out in the metatheory by Agda, as a recursion on the meta-natural r that turns the six-step block into a function $\text{Deriv}(S(r)) \rightarrow \text{Deriv}(S(r+1))$ and iterates it from the base derivation of $S(0)$ up to $S(N)$.

4 The formalisation

The proof is checked in Agda, in the same T4 development as the Guard-route proof of [1], under the global options `--safe --without-K --exact-split`, with no postulates and no holes. The headline is a single function from a derivation of Con_T to a derivation of $\mathbf{O} = \mathbf{s} \mathbf{O}$; Con_T is the only hypothesis, and it is the same open consistency formula as in [1].

The one genuinely non-classical piece is the Gandy–Howard majorant of §2.3. The diagonalisation is Chaitin’s, but the closed object term occupying the fuel slot of the internal interpreter run has no counterpart in the informal argument, and its construction (by recursion mirroring the program, with a substituted-motive induction through the recursion node) is what lets evaluator completeness be *stated and proved* inside BRA. It is worth noting, finally, that Guard’s formulation of Basic Recursive Arithmetic — syntax-first, with the verifier $\mathbf{thmT}$ and the numeral functor $\mathbf{num}$ as honest object functions — proved well suited to expressing all of these results, including the internal Chaitin implication and the surprise descent.

The development, including both proofs of the second incompleteness theorem, is available at <https://github.com/coquand/agda-godel-tree>.

References

- [1] T. Coquand. Autoformalisation of Gödel’s second incompleteness theorem in Basic Recursive Arithmetic. arXiv:2606.01898, 2026. <https://arxiv.org/abs/2606.01898>.
- [2] G. J. Chaitin. Computational complexity and Gödel’s incompleteness theorem. *ACM SIGACT News*, 9:11–12, 1971.
- [3] G. J. Chaitin. Information-theoretic limitations of formal systems. *Journal of the ACM*, 21(3):403–424, 1974.
- [4] R. O. Gandy. Proofs of strong normalization. In J. P. Seldin and J. R. Hindley, editors, *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, pages 457–477. Academic Press, 1980.
- [5] J. R. Guard. Lecture notes on recursive arithmetic. Unpublished lecture notes, 1963.
- [6] W. A. Howard. Hereditarily majorizable functionals of finite type. In A. S. Troelstra, editor, *Metamathematical Investigations of Intuitionistic Arithmetic and Analysis*, volume 344 of *Lecture Notes in Mathematics*, pages 454–461. Springer, 1973.
- [7] M. Kikuchi. Kolmogorov complexity and the second incompleteness theorem. *Archive for Mathematical Logic*, 36:437–443, 1997.
- [8] S. Kritchman and R. Raz. The surprise examination paradox and the second incompleteness theorem. *Notices of the American Mathematical Society*, 57(11):1454–1458, 2010.
- [9] J. R. Shoenfield. *Mathematical Logic*. Addison–Wesley, Reading, MA, 1967.